

PROJECT MANAGEMENT SYSTEM

A Mini Project Report

Submitted in partial fulfillment of requirement of the
Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER
SCIENCE & ENGINEERING**

BY

Muskaan Gupta, Muskan Kumari, Muskan Kulria
EN23CS301639, EN23CS301641, EN23CS301640

Under the Guidance of
Prof. Sonal Modh Bhardwaj
Prof. Rashmi Choudhary



Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

Report Approval

The project work "PROJECT MANAGEMENT SYSTEM" is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn there in; but approve the "Project Report" only for the purpose for which it has been submitted.

Internal Examiner 1
Prof. Rashmi Choudhary
Assistant Professor
Medicaps University

Internal Examiner 2
Prof. Sonal Modh
Assistant Professor
Medicaps University

External Examiner

Name:
Designation:
Affiliation:

Declaration

We hereby declare that the project entitled "**PROJECT MANAGEMENT SYSTEM**" submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science & Engineering completed under the supervision of **Prof. Sonal Modh Mentor, Prof. Rashmi Choudhary** Faculty of Engineering, Medi-Caps University Indore is an authentic work.

Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the student(s) with date:

Muskaan Gupta (EN23CS301639) _____

Muskan Kumari (EN23CS301641) _____

Muskan Kulria (EN23CS301640) _____

Certificate

We, **Prof. Sonal Modh Mentor** and **Prof. Rashmi Choudhary** , certify that the project entitled "**PROJECT MANAGEMENT SYSTEM**" submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science & Engineering by **Muskaan Gupta, Muskan Kumari, and Muskan Kulria** is the record carried out by them under my guidance and that the work has not formed the basis of award of any other degree elsewhere.

Prof. Rashmi Choudhary

Computer Science & Engineering

Medicaps University, Indore

Prof. Sonal Modh Bhardwaj

Computer Science & Engineering

Medicaps University, Indore

Dr. Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering

Medicaps University, Indore

Acknowledgements

We would like to express our deepest gratitude to the Honorable Chancellor, **Shri R C Mittal**, who has provided us with every facility to successfully carry out this project, and our profound indebtedness to **Prof. (Dr.) D. K. Patnaik**, Vice Chancellor, Medi-Caps University, whose unfailing support and enthusiasm has always boosted our morale. We also thank **Prof. (Dr.) Pramod S. Nair**, Dean, Faculty of Engineering, Medi-Caps University, for giving us a chance to work on this project. We would also like to thank our Head of the Department **Dr. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

We are deeply grateful to our project guide **Prof. Rashmi Choudhary, Prof. Sonal Modh Bhardwaj**, Department of Computer Science & Engineering, Medi-Caps University, Indore, for her valuable guidance, constructive suggestions, constant motivation, and unwavering support throughout the development of this project. Her technical expertise and patient mentorship have been instrumental in shaping the outcome of this work.

We also extend our sincere thanks to all the faculty members and staff of the Department of Computer Science & Engineering who provided us with the necessary resources and a conducive academic environment. Their insights and feedback during technical discussions significantly contributed to the quality of this project.

Muskan Kulria

Muskan Kumari

Muskaan Gupta

B.Tech. III Year (K)

Department of Computer Science & Engineering

Faculty of Engineering

Medicaps University, Indore

Abstract

The Project Management System (PMS) is a comprehensive web-based application designed to facilitate efficient planning, execution, monitoring, and closure of projects within an organizational setting. In the modern digital era, organizations are increasingly dependent on technology-driven solutions to manage their projects, teams, and resources effectively. This system addresses the critical need for a centralized platform that allows project managers and team members to collaborate seamlessly, track progress in real-time, and make informed decisions based on data-driven insights.

The proposed system has been developed using a modern full-stack technology architecture comprising ReactJS for the frontend user interface, FastAPI as the backend framework, and MongoDB as the NoSQL database. ReactJS enables the development of a dynamic, responsive, and user-friendly interface through component-based architecture and efficient state management. FastAPI, being a high-performance Python-based framework, ensures rapid API development with automatic data validation, serialization, and comprehensive documentation support via OpenAPI standards. MongoDB, as a document-oriented database, provides flexible schema design suitable for managing the diverse and evolving data requirements of project management.

The system encompasses several key modules including user authentication and role-based access control, project creation and configuration, task assignment and management, milestone tracking, team collaboration, file sharing, notification management, and reporting and analytics. The role-based access control module ensures that system administrators, project managers, and team members can interact with the application within defined permission boundaries, thereby maintaining data security and operational integrity.

The task management module enables the creation, assignment, prioritization, and tracking of individual tasks within a project. Each task can be associated with deadlines, dependencies, assigned team members, and completion statuses such as To Do, In Progress, Under Review, and Completed. The system also supports milestone tracking, which helps project stakeholders monitor the progress of significant deliverables against predefined timelines.

A real-time notification system ensures that team members are promptly informed of any updates, such as new task assignments, deadline reminders, status changes, or comments on their work items. The reporting module generates visual dashboards and statistical summaries that provide project managers with a clear overview of project health, resource utilization, task completion rates, and potential risks.

The system has been developed following standard software engineering practices including requirements analysis, system design using UML and Data Flow Diagrams, iterative implementation, and comprehensive testing through unit, integration, and user acceptance testing methodologies. The implementation ensures that the application is scalable, maintainable, and accessible across different devices and screen sizes through responsive design principles.

The Project Management System represents a practical solution for academic institutions, small businesses, and development teams seeking to improve their project coordination and delivery capabilities. This report presents the complete documentation of the system from inception to deployment, covering all aspects of software engineering methodology.

Keywords: Project Management System, ReactJS, FastAPI, MongoDB, Task Management, Role-Based Access Control, Web Application, REST API, Agile, Full-Stack Development, NoSQL, Real-Time Notifications, Software Engineering, Dashboard Analytics

Table of Contents

Chapter	Title	Page No
	Report Approval	ii
	Declaration	iii
	Certificate	iv
	Acknowledgement	v
	Abstract	vi
	Table of Contents	viii
	List of Figures	x
	List of Tables	xi
	Abbreviations	xii
	Notations & Symbols	xiii
Chapter 1	Introduction	
	1.1 Introduction	1
	1.2 Literature Review	2
	1.3 Objectives	4
	1.4 Significance	4
	1.5 Research Design	5
	1.6 Source of Data	5
	1.7 Chapter Scheme	6
Chapter 2	Requirements Specification	
	2.1 User Characteristics	7
	2.2 Functional Requirements	7
	2.3 Dependencies	9
	2.4 Performance Requirements	9
	2.5 Hardware Requirements	9
	2.6 Constraints & Assumptions	10
Chapter 3	Design	
	3.1 Algorithm	11
	3.2 Function Oriented Design	12

	3.3 System Design	12
	3.3.1 Data Flow Diagrams (Level 0, Level 1)	13
	3.3.2 Activity Diagram	15
	3.3.3 Flow Chart	17
	3.3.4 Class Diagram	18
	3.3.5 ER Diagram	19
	3.4 Database Design	20
	3.4.1 Logical Database Design	20
	3.4.2 Physical Database Design	21
Chapter 4	Implementation, Testing, and Maintenance	
	4.1 Introduction to Languages, IDEs, Tools and Technologies	22
	4.2 Testing Techniques and Test Plans	24
	4.3 Installation Instructions	26
	4.4 End User Instructions	27
Chapter 5	Results and Discussions	
	5.1 User Interface Representation	29
	5.2 Brief Description of Various Modules	30
	5.3 Snapshots of System with Brief Detail	31
	5.4 Back End Representation	33
Chapter 6	Summary and Conclusions	35
Chapter 7	Future Scope	37
	Appendix	39
	Bibliography	40

List of Figures

Figure No.	Figure Title
Fig. 3.1	Data Flow Diagram - Level 0 (Context Diagram)
Fig. 3.2	Data Flow Diagram - Level 1
Fig. 3.3	Activity Diagram for Task Management
Fig. 3.4	Flowchart for User Login Process
Fig. 3.5	Class Diagram of Project Management System
Fig. 3.6	Entity Relationship Diagram
Fig. 3.7	Sequence Diagram for Task Assignment
Fig. 5.1	Login/Registration UI Screen
Fig. 5.2	Admin Dashboard UI
Fig. 5.3	Project Creation Screen
Fig. 5.4	Task Board (Kanban View)
Fig. 5.5	Reports and Analytics Dashboard

List of Tables

Table No.	Table Title
Table 2.1	Functional Requirements Summary
Table 2.2	Non-Functional Requirements
Table 2.3	Hardware and Software Requirements
Table 3.1	MongoDB Collections and Fields - Users
Table 3.2	MongoDB Collections and Fields - Projects
Table 3.3	MongoDB Collections and Fields - Tasks
Table 3.4	MongoDB Collections and Fields - Notifications
Table 4.1	Tools and Technologies Used
Table 4.2	Unit Test Cases
Table 4.3	Integration Test Cases
Table 4.4	User Acceptance Test Cases
Table 5.1	Database Collection Sample Documents

Abbreviations

PMS	Project Management System
API	Application Programming Interface
REST	Representational State Transfer
UI	User Interface
UX	User Experience
CRUD	Create, Read, Update, Delete
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model View Controller
NoSQL	Not Only Structured Query Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
DFD	Data Flow Diagram
ER	Entity Relationship
UML	Unified Modeling Language
IDE	Integrated Development Environment
ODM	Object Document Mapper
RBAC	Role-Based Access Control
SRS	Software Requirements Specification
UAT	User Acceptance Testing

Notations & Symbols

Symbol / Notation	Meaning
{ }	Denotes a MongoDB document (JSON object)
[]	Denotes an array of documents or values
_id	Unique identifier field in MongoDB collection
->	Refers to a reference or relationship between collections
*	Denotes a mandatory / required field
~	Approximately equal to
N	Number of items in a collection or dataset
O(n)	Time complexity notation (linear)
O(1)	Time complexity notation (constant)
<<include>>	UML notation for include relationship in use cases
<<extend>>	UML notation for extend relationship in use cases

Chapter-1

INTRODUCTION

1.1 Introduction

In today's fast-paced technological environment, the ability to manage projects efficiently is a critical determinant of organizational success. A Project Management System (PMS) serves as a centralized digital platform that brings together all aspects of project planning, execution, monitoring, and closure under a unified interface. It enables organizations of varying sizes to coordinate tasks, manage teams, allocate resources, track milestones, and generate comprehensive reports, thereby facilitating better decision-making and improved project outcomes.

Traditional project management methods relying on spreadsheets, physical boards, or disconnected tools suffer from significant limitations including lack of real-time updates, poor collaboration capabilities, data redundancy, and absence of comprehensive analytics. These shortcomings necessitate the development of a dedicated, technology-driven solution that addresses all aspects of project management in an integrated and scalable manner.

The Project Management System described in this report is a full-stack web application built using a modern technology stack. The frontend is implemented using ReactJS, a popular JavaScript library developed by Meta, which enables the creation of interactive and responsive user interfaces through a component-based architecture. The backend is powered by FastAPI, a high-performance Python framework that facilitates the development of RESTful APIs with automatic data validation, serialization, and OpenAPI documentation. MongoDB, a document-oriented NoSQL database, has been selected for data persistence due to its flexible schema design, horizontal scalability, and compatibility with JSON-like documents.

The system supports multiple user roles including Administrator, Project Manager, and Team Member, each with defined access permissions and capabilities. Project managers can create and configure projects, define tasks and milestones, assign responsibilities to team members, and monitor overall progress through an intuitive dashboard. Team members can view their assigned tasks, update statuses, upload deliverables, and communicate within the system. Administrators maintain system configurations, manage user accounts, and oversee all ongoing projects.

This report presents the complete software engineering documentation of the Project Management System, covering requirements specification, system and database design, implementation details, testing methodologies, and a discussion of results. The work has been undertaken as a B.Tech Mini Project by students of the Department of Computer Science & Engineering, Medi-Caps University, Indore, under the supervision of Prof. Sonal Modh.

1.2 Literature Review

Project management as a discipline has been studied extensively by researchers across various domains. The evolution of project management software from early desktop-based tools to cloud-based, collaborative platforms reflects the growing demand for integrated and accessible solutions. This section presents a review of relevant literature pertaining to project management systems, web application development, and the technologies employed in this project.

Kerzner (2017) in his seminal work on project management defines a project management information system as a computer-driven tool that collects, integrates, and reports project information to assist in planning and controlling projects. He emphasizes the importance of real-time data visibility and stakeholder communication as key factors in project success. These principles have directly influenced the design of the notification and reporting modules of the present system.

Sommerville (2016) in his textbook on Software Engineering discusses the importance of requirements engineering and iterative development methodologies in ensuring that software systems meet user needs effectively. His framework for functional and non-functional requirements specification has been adopted in Chapter 2 of this report. Sommerville also advocates for the use of UML notations for system modeling, which has been applied in the design chapter.

Research by Baccarini (1996) on project complexity and project success criteria establishes that a clear definition of project scope, responsibilities, and timelines significantly reduces project failure rates. This research supports the inclusion of milestone tracking and role-based access control features in the proposed system, ensuring that scope creep and responsibility overlaps are minimized.

In the context of web technologies, Fedosejev (2015) provides a comprehensive guide to ReactJS application development, describing the advantages of the virtual DOM, unidirectional data flow, and the reusability of components. These architectural benefits of

ReactJS make it particularly suitable for building complex, data-driven interfaces such as project dashboards and kanban boards. The component-based structure of ReactJS has been leveraged extensively in the frontend design of this system.

FastAPI, introduced by Ramirez (2018), represents a significant advancement in Python-based API development. Studies comparing FastAPI with Flask and Django REST Framework have consistently demonstrated FastAPI's superior performance, particularly in terms of request handling speed and developer productivity. The automatic generation of OpenAPI documentation is a notable advantage that simplifies the integration between frontend and backend components. This feature has been particularly beneficial during the development and testing phases of the present project.

Chodorow (2013) in the comprehensive guide to MongoDB highlights the advantages of document-oriented databases for applications with evolving data models. The flexible schema design of MongoDB allows development teams to adapt data structures without requiring complex migrations, which is highly relevant for project management applications where the attributes of projects and tasks may vary significantly across different use cases. The use of MongoDB's aggregation pipeline has enabled efficient generation of project statistics and reports in this system.

Research by Kniberg and Skarin (2010) on Kanban and Scrum methodologies underlines the effectiveness of visual task boards in improving team productivity and transparency. Their work has inspired the implementation of the Kanban-style task board in the present system, where tasks are visually organized into columns representing different stages of completion. This visual representation enhances team awareness and facilitates daily stand-up meetings.

A review of existing project management tools such as Jira (Atlassian), Trello, Asana, and Microsoft Project reveals that while these tools offer comprehensive features, they are often proprietary, expensive, and may not be readily customizable for specific organizational workflows. The development of a custom project management system using open-source technologies provides greater flexibility, cost efficiency, and the opportunity to align the system precisely with the requirements of the target organization.

1.3 Objectives

The primary objectives of the Project Management System are defined as follows:

- To design and develop a fully functional web-based Project Management System using ReactJS, FastAPI, and MongoDB.
- To implement a secure user authentication module with JSON Web Token (JWT)-based authorization and role-based access control for Administrators, Project Managers, and Team Members.
- To provide project managers with tools to create, configure, and manage multiple projects simultaneously, including the definition of project scope, timelines, and budgets.
- To facilitate task creation, assignment, prioritization, and tracking with support for task dependencies, deadlines, and status updates.
- To implement milestone tracking to enable monitoring of critical project deliverables against predefined schedules.
- To develop a real-time notification system that alerts users about task assignments, deadline approaches, status changes, and system events.
- To generate interactive reports and dashboards that provide visual insights into project progress, resource utilization, and team performance.
- To support file upload and management capabilities for sharing project-related documents and deliverables within the system.
- To ensure the system is responsive, accessible, and compatible across modern web browsers and devices.
- To implement comprehensive testing strategies including unit testing, integration testing, and user acceptance testing to validate system functionality and reliability.

1.4 Significance

The significance of the Project Management System lies in its ability to address the practical challenges faced by development teams and organizations in managing complex projects. By centralizing project information, task assignments, and communication within a single platform, the system eliminates the need for multiple disconnected tools and reduces the administrative overhead associated with manual project tracking.

From an academic perspective, the development of this system provides the student team with hands-on experience in full-stack web application development, database design, RESTful API architecture, and software testing. The project demonstrates the practical application of computer science concepts including data structures, algorithms, software engineering principles, and web technologies.

For Medi-Caps University and similar academic institutions, the system can serve as a practical tool for managing student projects, faculty research initiatives, and departmental

activities. Its open-source technology stack ensures that the system can be deployed and maintained without incurring significant licensing costs.

1.5 Research Design

The research design adopted for this project follows the Software Development Life Cycle (SDLC) model, specifically an iterative and incremental approach inspired by Agile principles. The development process was organized into the following phases: requirements gathering, system analysis, design, implementation, testing, and documentation.

Requirements were gathered through consultation with potential users, review of existing project management tools, and analysis of academic literature. Use cases were identified and documented to capture the functional requirements of the system. Non-functional requirements relating to performance, security, scalability, and usability were also defined during this phase.

The system design phase involved the creation of architectural diagrams including Data Flow Diagrams, Entity Relationship Diagrams, Class Diagrams, and Sequence Diagrams. The database schema was designed to leverage MongoDB's document model effectively, with collections defined for users, projects, tasks, milestones, notifications, and files.

Implementation followed a modular approach, with the frontend and backend developed in parallel. The ReactJS frontend was structured using a component hierarchy with shared state managed through React Context API and hooks. The FastAPI backend was organized into routers, models, schemas, and database utility modules following best practices for project structure and code maintainability.

1.6 Source of Data

The data used in the development and testing of the Project Management System is primarily synthetic, generated specifically for the purpose of populating the application with realistic test scenarios. Sample user accounts representing different roles, projects with varying complexities, tasks at different stages of completion, and milestone records have been created to validate the functionality of each module.

For the literature review, data has been sourced from academic textbooks, peer-reviewed journal articles, official documentation of the technologies used (React, FastAPI, MongoDB), and reputable online resources including the Mozilla Developer Network (MDN),

PyPI documentation, and MongoDB's official documentation portal. All sources have been properly cited in the bibliography section of this report following the IEEE citation format.

1.7 Chapter Scheme

The report is organized into seven chapters as described below:

Chapter 1 - Introduction: Presents the background, motivation, literature review, objectives, significance, research design, and chapter overview of the project.

Chapter 2 - Requirements Specification: Details the functional and non-functional requirements of the system, user characteristics, hardware and software dependencies, performance requirements, and system constraints.

Chapter 3 - Design: Covers the system architecture, Data Flow Diagrams at Level 0 and Level 1, Activity Diagrams, Flowcharts, Class Diagrams, ER Diagram, Sequence Diagrams, and database design including logical and physical schemas.

Chapter 4 - Implementation, Testing, and Maintenance: Describes the tools and technologies used, implementation details of the ReactJS frontend and FastAPI backend, testing techniques and test plans, installation procedures, and end user instructions.

Chapter 5 - Results and Discussions: Presents the user interface descriptions, module summaries, system snapshots with detailed descriptions, and backend database representations.

Chapter 6 - Summary and Conclusions: Summarizes the work carried out, presents the conclusions drawn from the project, and discusses the overall achievement of objectives.

Chapter 7 - Future Scope: Identifies potential enhancements and future directions for the system, including advanced features, integrations, and scalability improvements.

Chapter-2

REQUIREMENTS SPECIFICATION

2.1 User Characteristics

The Project Management System is designed to serve three distinct categories of users, each with specific roles, responsibilities, and system interaction patterns.

Administrator: The administrator is a technically proficient user responsible for the overall management of the system. This includes creating and managing user accounts, assigning roles and permissions, monitoring system health, and configuring global system settings. The administrator has access to all modules and data within the system without restriction. It is assumed that the administrator has a good understanding of system administration principles and is comfortable using web-based dashboards.

Project Manager: The project manager is a professional user responsible for planning, executing, and monitoring projects within the system. This user is expected to have familiarity with project management concepts including task decomposition, milestone planning, resource allocation, and progress reporting. The project manager can create projects, define tasks and milestones, assign team members, set deadlines, and view comprehensive project reports and analytics.

Team Member: Team members are end users who participate in project execution under the direction of a project manager. They are expected to have basic computer literacy and familiarity with web browsers. Team members can view their assigned tasks, update task statuses, upload files, and communicate within the system. They do not have access to project configuration or administrative settings.

2.2 Functional Requirements

The functional requirements define the specific behaviors and capabilities that the system must provide to its users. The following functional requirements have been identified through analysis and user consultation:

2.2.1 User Authentication and Authorization

- The system shall allow new users to register with a valid email address, username, and password.
- The system shall authenticate users using email and password credentials.

- The system shall implement JWT-based session management to maintain user sessions securely.
- The system shall enforce role-based access control, restricting access to features based on the user's assigned role.
- The system shall provide a password reset mechanism via registered email address.

2.2.2 Project Management

- The system shall allow project managers to create new projects with a title, description, start date, end date, and priority level.
- The system shall allow project managers to edit project details, archive projects, and delete projects.
- The system shall support the assignment of multiple team members to a project.
- The system shall display a list of all projects with their current status (Active, On Hold, Completed, Cancelled).

2.2.3 Task Management

- The system shall allow project managers to create tasks within a project with a title, description, assignee, priority, start date, due date, and status.
- The system shall support task status updates: To Do, In Progress, Under Review, and Completed.
- The system shall allow team members to update the status of tasks assigned to them.
- The system shall support task dependencies to indicate prerequisite relationships.
- The system shall provide a Kanban board view for visualizing task progress.

2.2.4 Milestone Tracking

- The system shall allow the creation of milestones associated with specific projects.
- The system shall enable tracking of milestone completion against target dates.
- The system shall display milestone progress using a visual timeline representation.

2.2.5 Notifications

- The system shall generate notifications for task assignments, deadline reminders, status changes, and comments.
- The system shall display unread notification count in the interface header.
- The system shall allow users to mark notifications as read or delete them.

2.2.6 Reporting and Analytics

- The system shall generate project summary reports showing task completion rates, overdue tasks, and milestone status.
- The system shall provide graphical charts for visualizing project progress and team performance.
- The system shall allow export of reports in a printable format.

2.2.7 File Management

- The system shall allow users to upload files (documents, images) to specific tasks or projects.

- The system shall display a list of uploaded files with their names, upload dates, and uploader information.

2.3 Dependencies

The Project Management System has the following external software and library dependencies:

- Node.js (v18 or above) and npm for managing the ReactJS frontend build process.
- Python (v3.10 or above) as the runtime environment for the FastAPI backend.
- MongoDB (v6.0 or above) as the primary database system.
- PyMongo or Motor library for asynchronous MongoDB interaction from the FastAPI backend.
- Uvicorn ASGI server for running the FastAPI application.
- Pydantic library for data validation and schema definition in FastAPI.
- React Router DOM for client-side routing in the ReactJS application.
- Axios library for making HTTP requests from the React frontend to the FastAPI backend.
- JWT (python-jose) library for encoding and decoding JSON Web Tokens.
- Passlib library with bcrypt hashing for secure password storage.
- Chart.js or Recharts library for rendering graphical dashboards in React.

2.4 Performance Requirements

- The system shall respond to API requests within 2 seconds under normal load conditions (up to 100 concurrent users).
- The login and authentication process shall complete within 1 second.
- The dashboard and project list pages shall load within 3 seconds on a standard broadband connection.
- The system shall support a minimum of 500 concurrent users without significant degradation in performance.
- Database query response times shall not exceed 500 milliseconds for standard CRUD operations.
- File uploads shall support files up to 10 MB in size with an upload time not exceeding 10 seconds.

2.5 Hardware Requirements

The following hardware requirements apply for deploying the Project Management System:

Component	Minimum Requirement
Server CPU	Intel Core i5 or equivalent (2 GHz, 4 cores)
Server RAM	8 GB DDR4
Storage	50 GB SSD (for application and database)

Network	100 Mbps broadband internet connection
Client Device	Any modern device with a web browser (PC, laptop, tablet, smartphone)
Browser Support	Chrome (v90+), Firefox (v88+), Edge (v90+), Safari (v14+)

2.6 Constraints & Assumptions

The following constraints and assumptions govern the development and deployment of the system:

- The system requires an active internet connection for all operations, as it is designed as a web-based application.
- It is assumed that all users have a valid email address for account creation and password recovery.
- The system currently supports only the English language; multilingual support is not included in the current scope.
- File storage is handled locally on the server in the current version; cloud storage integration (e.g., AWS S3) is identified as future scope.
- The system does not include video conferencing or chat messaging capabilities in the current version.
- It is assumed that the MongoDB instance is either locally hosted or accessible via a MongoDB Atlas cloud cluster.
- The development team assumes that all users will access the system using a device with a screen resolution of at least 1024x768 pixels.
- The system does not currently support offline operation; all data access and synchronization require an active server connection.

Chapter-3

DESIGN

3.1 Algorithm

The core algorithmic logic of the Project Management System involves several computational processes that govern task assignment, status transitions, notification generation, and report computation. The following pseudocode algorithms describe the primary logic flows:

3.1.1 User Authentication Algorithm

The authentication algorithm follows a challenge-response mechanism using JWT tokens. When a user submits credentials, the system retrieves the corresponding user document from the MongoDB users collection using the provided email. The stored hashed password is compared against the submitted password using bcrypt comparison. If authentication succeeds, a JWT access token is generated encoding the user's ID and role with an expiry timestamp. Subsequent requests include this token in the Authorization header, which the FastAPI dependency injection system validates before processing each protected endpoint.

3.1.2 Task Assignment and Notification Algorithm

When a project manager creates or updates a task with an assigned user, the system first verifies that the assigned user is a member of the parent project. Upon successful verification, the task document is created or updated in the tasks collection. The system then creates a notification document in the notifications collection with the recipient's user ID, a descriptive message, the task reference, and a timestamp. The unread notification count for the affected user is incremented, which triggers a real-time update in the frontend through API polling.

3.1.3 Project Progress Computation Algorithm

The project progress is computed using MongoDB's aggregation pipeline. For a given project ID, the system queries all associated tasks and groups them by status using the \$group aggregation operator. The total task count and counts for each status category (To Do, In Progress, Under Review, Completed) are computed. The completion percentage is calculated as (Completed Tasks / Total Tasks) multiplied by 100. This aggregated result is returned as part of the project summary response and rendered as a progress bar and pie chart on the frontend dashboard.

3.2 Function Oriented Design for Procedural Approach

The Project Management System follows a function-oriented modular design at the backend level. The FastAPI application is organized into distinct router modules, each responsible for a specific functional domain. The main functional modules and their primary functions are described below:

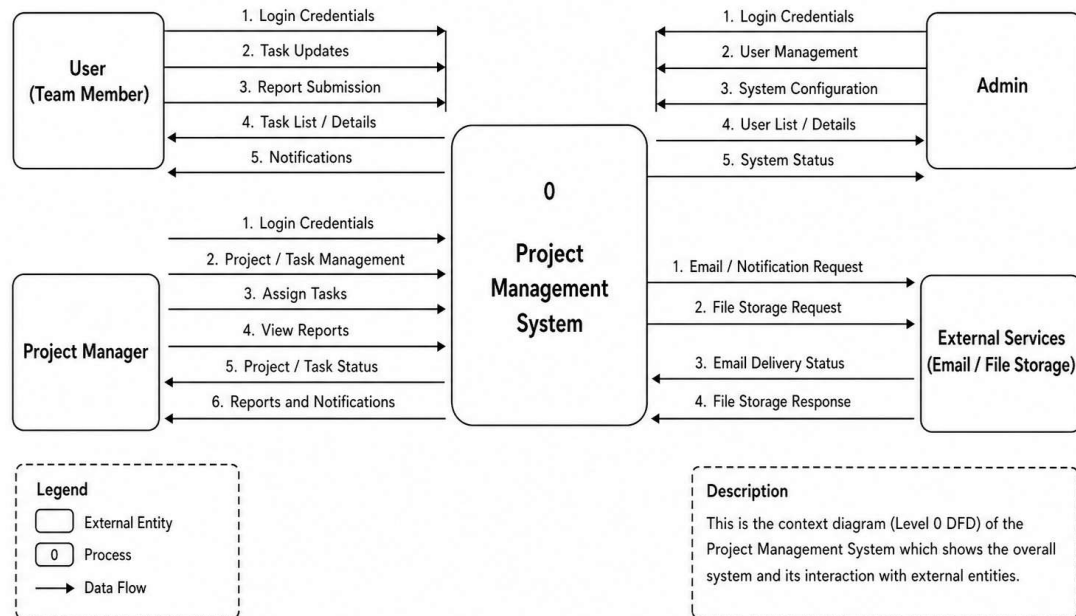
- **auth_router:** Functions include `register_user()`, `login_user()`, `refresh_token()`, `reset_password()`. These functions handle user registration, credential verification, token generation, and password management.
- **project_router:** Functions include `create_project()`, `get_projects()`, `update_project()`, `delete_project()`, `add_member()`. These functions manage the full lifecycle of projects within the system.
- **task_router:** Functions include `create_task()`, `get_tasks()`, `update_task_status()`, `assign_task()`, `delete_task()`. These functions handle task creation, retrieval, status management, and deletion.
- **milestone_router:** Functions include `create_milestone()`, `get_milestones()`, `update_milestone()`, `delete_milestone()`. These functions support milestone management within projects.
- **notification_router:** Functions include `get_notifications()`, `mark_as_read()`, `delete_notification()`. These functions manage the notification lifecycle for users.
- **report_router:** Functions include `get_project_summary()`, `get_task_statistics()`, `get_user_performance()`. These functions execute aggregation queries and return report data to the frontend.
- **file_router:** Functions include `upload_file()`, `get_files()`, `delete_file()`. These functions manage file uploads and retrieval associated with tasks and projects.

3.3 System Design

3.3.1 Data Flow Diagrams (Level 0 and Level 1)

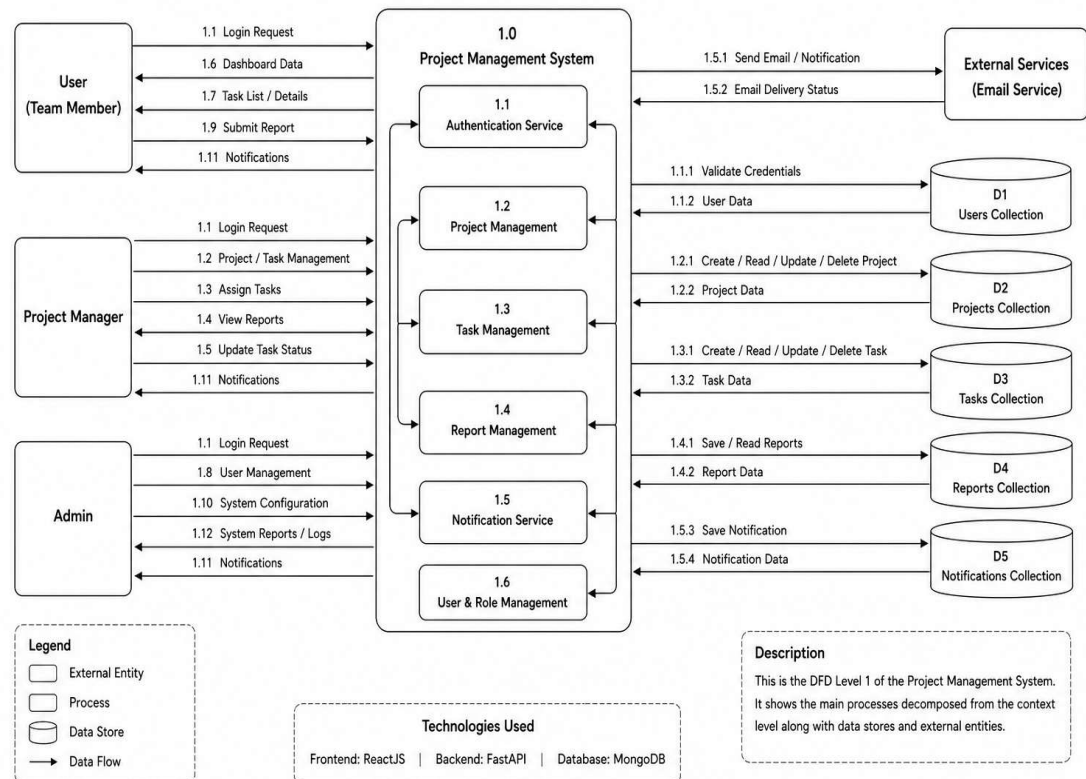
Data Flow Diagrams (DFDs) represent the flow of data through the system, showing how data is input, processed, and output across different components. DFDs are constructed using standard notation including external entities (rectangles), processes (circles or rounded rectangles), data stores (open-ended rectangles), and data flows (arrows).

DFD Level 0 (Context Diagram) – Project Management System



Level 0 DFD (Context Diagram): The context diagram represents the highest level of abstraction for the Project Management System. It depicts the entire system as a single process labeled 'Project Management System' at the center. Three external entities interact with this central process: the User (which encompasses the roles of Administrator, Project Manager, and Team Member), the Notification Service (representing external communication triggers), and the File Storage (representing the data persistence layer for uploaded files). Data flows from the User entity into the system include Login Credentials, Project Data, Task Data, File Uploads, and Report Requests. Data flows from the system back to the User include Authentication Tokens, Project Listings, Task Assignments, Notifications, and Generated Reports. The system also exchanges data with the File Storage entity through file upload and retrieval operations.

DFD Level 1 – Project Management System



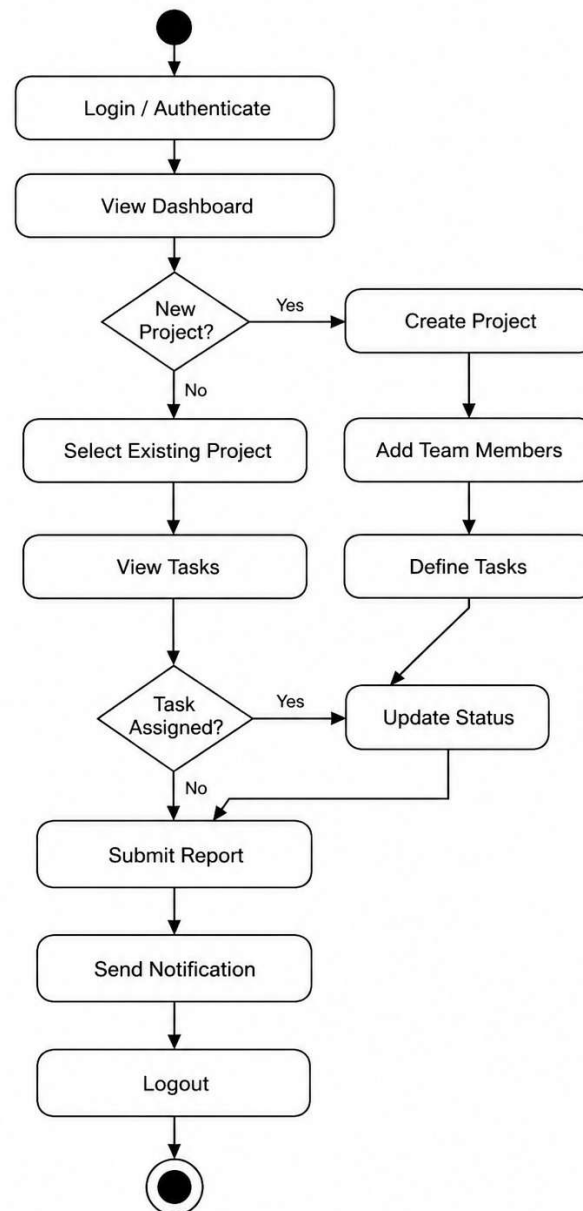
Level 1 DFD: The Level 1 DFD decomposes the central process of the context diagram into its constituent functional processes, revealing the internal data flows between them. The Level 1 DFD includes the following processes and their interactions:

- **Process 1.0 - User Authentication:** Receives user credentials from the external User entity, interacts with the Users data store (D1) to verify credentials, and outputs authentication tokens back to the user.
- **Process 2.0 - Project Management:** Receives project creation and update requests from authenticated project managers, reads and writes to the Projects data store (D2), and outputs project data to authorized users.
- **Process 3.0 - Task Management:** Receives task creation, assignment, and update requests, interacts with the Tasks data store (D3) and the Users data store (D1) for assignment validation, and outputs task details and status updates.
- **Process 4.0 - Milestone Tracking:** Receives milestone data from project managers, reads and writes to the Milestones data store (D4), and provides milestone progress information.
- **Process 5.0 - Notification Management:** Receives triggers from Process 3.0 (task assignment), Process 4.0 (milestone updates), and other processes, writes notification records to the Notifications data store (D5), and delivers notification data to the relevant users.

- Process 6.0 - Report Generation: Reads from the Projects data store (D2), Tasks data store (D3), and Milestones data store (D4) using aggregation queries, and outputs report data and analytics to project managers.
- Process 7.0 - File Management: Receives file upload requests from users, stores files to the file system, writes metadata to the Files data store (D6), and retrieves file information on request.

3.3.2 Activity Diagram

Activity Diagram - Project Task Workflow



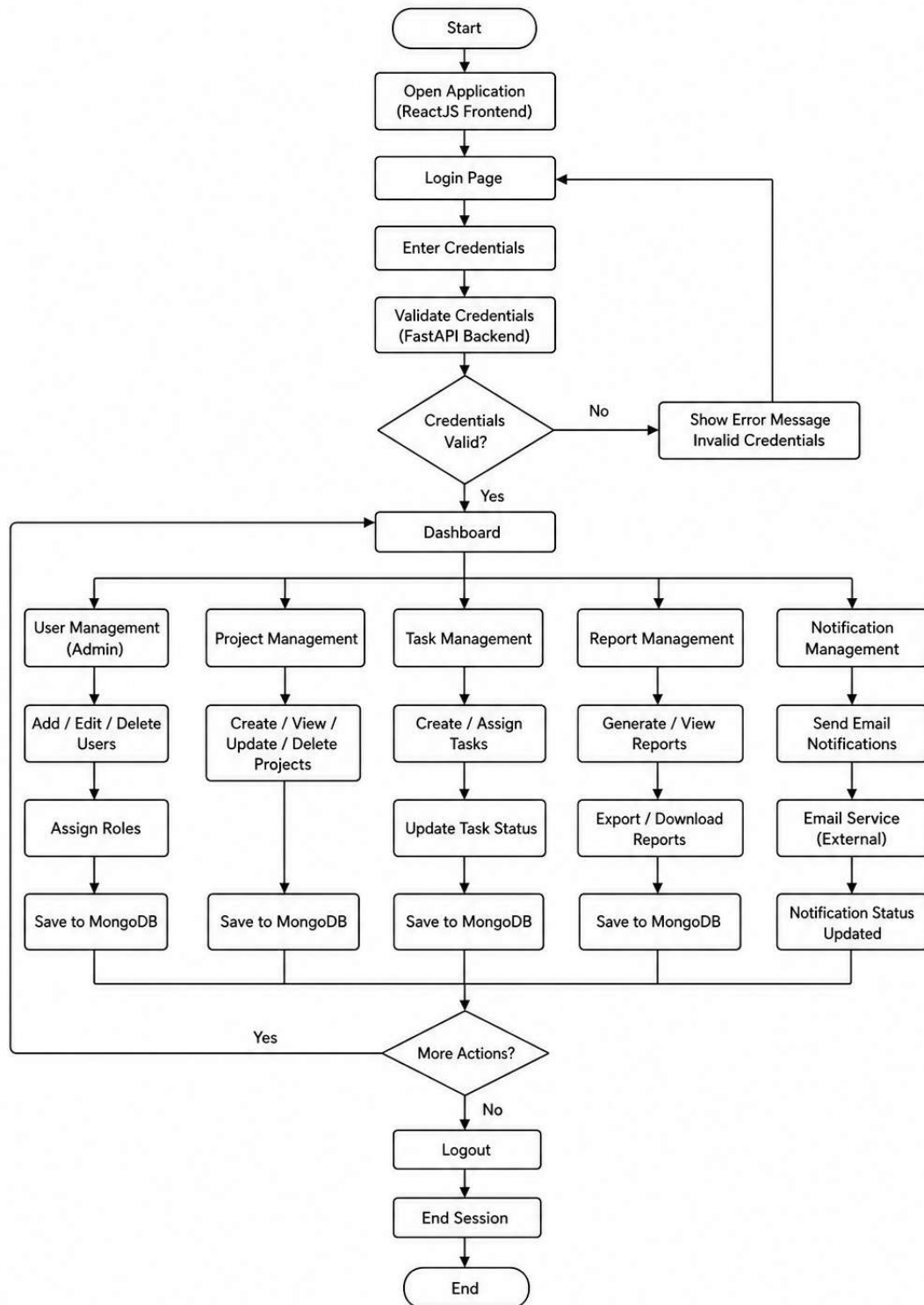
The Activity Diagram for the Task Management workflow describes the sequence of activities performed by a project manager and a team member in the context of task lifecycle management. The diagram begins with the start node and flows into the project manager's initial action of selecting a project from the dashboard. The project manager then creates a new task by entering the task title, description, priority, due date, and selecting an assignee from the list of project members. Upon saving, the system validates the input and creates the task record in the database. A parallel activity node branches the flow into two simultaneous activities: storing the task in the Tasks collection and generating a notification for the assigned team member. The assigned team member receives the notification, views the task details, and begins work, transitioning the task status to In Progress. Upon completion, the team member updates the status to Completed, which triggers another notification to the project manager. The project manager reviews the completed task, optionally provides feedback as a comment, and the activity diagram concludes with the task closure and project progress update.

3.3.3 Flow Chart

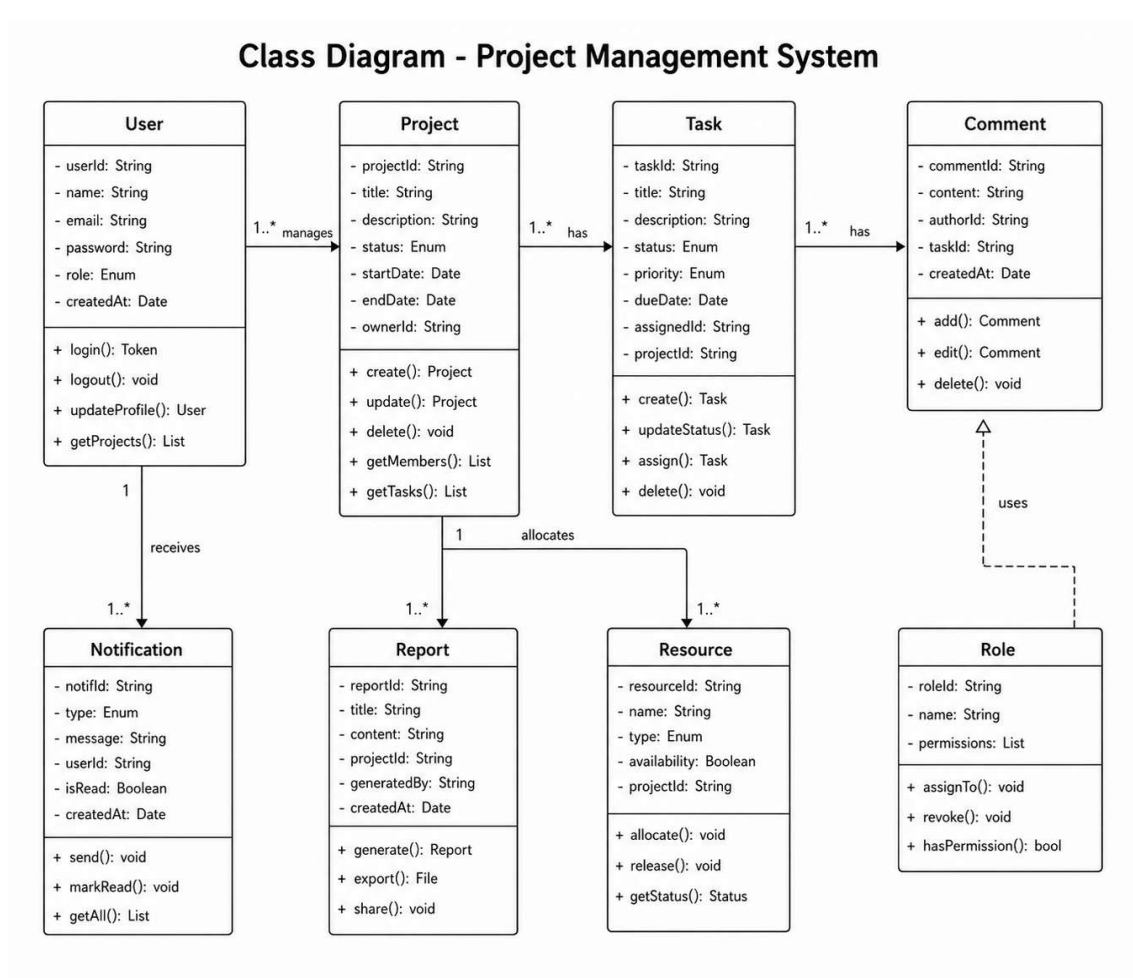
The flowchart for the User Login and Dashboard Routing process is described as follows. The process begins with the user entering the application URL. The system checks whether a valid JWT token exists in the browser's local storage. If no token is found, the user is redirected to the login page. On the login page, the user submits their email and password.

The system sends a POST request to the /auth/login API endpoint. A decision node checks whether the API response is successful (HTTP 200). If unsuccessful, an error message is displayed and the process loops back to the input fields. If successful, the JWT token returned in the response is stored in local storage. A second decision node evaluates the user's role extracted from the decoded JWT payload. If the role is 'admin', the user is directed to the Admin Dashboard. If the role is 'project_manager', the user is directed to the Project Manager Dashboard. If the role is 'team_member', the user is directed to the Team Member Dashboard. The flowchart concludes at the respective dashboard entry point.

Flowchart – Project Management System



3.3.4 Class Diagram



User class: Contains attributes user_id (String, primary identifier), username (String), email (String, unique), password_hash (String), role (Enum: admin, project_manager, team_member), created_at (DateTime), and is_active (Boolean). Methods include authenticate(), update_profile(), and get_assigned_tasks().

Project class: Contains attributes project_id (String), title (String), description (String), status (Enum: active, on_hold, completed, cancelled), priority (Enum: low, medium, high), start_date (Date), end_date (Date), created_by (User reference), and members (List of User references). Methods include add_member(), remove_member(), get_progress(), and get_tasks().

Task class: Contains attributes task_id (String), title (String), description (String), project_id (Project reference), assigned_to (User reference), priority (Enum: low, medium, high, critical), status (Enum: todo, in_progress, under_review, completed), start_date (Date),

due_date (Date), and created_at (DateTime). Methods include update_status(), assign_to_user(), and get_comments().

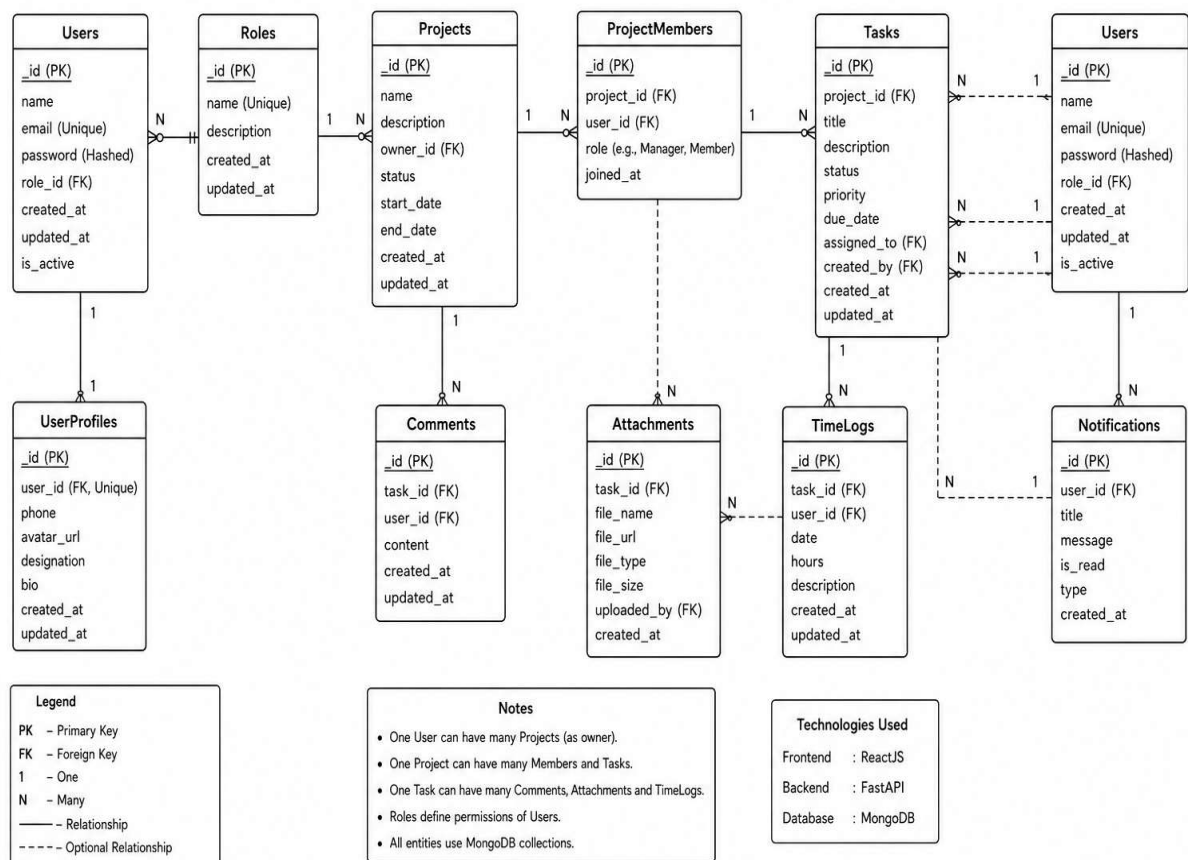
Notification class: Contains attributes notification_id (String), recipient_id (User reference), message (String), is_read (Boolean), created_at (DateTime), and reference_type (Enum: task, project, milestone). Methods include mark_as_read() and delete().

File class: Contains attributes file_id (String), file_name (String), file_path (String), uploaded_by (User reference), associated_task_id (Task reference), file_size (Integer), and uploaded_at (DateTime). Methods include get_download_url() and delete_file().

Relationships: User has a one-to-many relationship with Task (a user can be assigned many tasks). Project has a one-to-many relationship with Task and Milestone. User has a many-to-many relationship with Project through project membership. Notification has a many-to-one relationship with User (many notifications can belong to one user).

3.3.5 ER Diagram

ER Diagram – Project Management System



3.4 Database Design

3.4.1 Logical Database Design

The logical database design describes the structure of data in terms of collections, document schemas, and relationships, independent of the physical storage implementation. MongoDB stores data as BSON (Binary JSON) documents organized into collections. The following collections constitute the logical database design of the Project Management System:

Users Collection: Each document in the users collection represents a registered user of the system. The document schema includes: `_id` (ObjectId, auto-generated), `username` (String, required), `email` (String, required, unique index), `password_hash` (String, required), `role` (String, enum: admin/project_manager/team_member), `is_active` (Boolean, default: true), `created_at` (DateTime), `updated_at` (DateTime), `profile_picture` (String, optional URL).

Projects Collection: Each document represents a project. The schema includes: `_id` (ObjectId), `title` (String, required), `description` (String), `status` (String, enum), `priority` (String, enum), `start_date` (Date), `end_date` (Date), `created_by` (ObjectId, reference to users), `members` (Array of ObjectIds, references to users), `created_at` (DateTime), `updated_at` (DateTime), `budget` (Number, optional).

Tasks Collection: Each document represents a task within a project. The schema includes: `_id` (ObjectId), `title` (String, required), `description` (String), `project_id` (ObjectId, reference to projects), `assigned_to` (ObjectId, reference to users), `created_by` (ObjectId, reference to users), `priority` (String, enum), `status` (String, enum: todo/in_progress/under_review/completed), `start_date` (Date), `due_date` (Date, required), `comments` (Array of embedded comment documents), `tags` (Array of Strings), `created_at` (DateTime), `updated_at` (DateTime). Each embedded comment document contains: `comment_id` (ObjectId), `author_id` (ObjectId), `content` (String), `created_at` (DateTime).

Milestones Collection: Schema includes: `_id` (ObjectId), `title` (String, required), `description` (String), `project_id` (ObjectId), `due_date` (Date, required), `status` (String, enum: pending/completed), `completed_at` (DateTime, optional), `created_at` (DateTime).

Notifications Collection: Schema includes: `_id` (ObjectId), `recipient_id` (ObjectId, reference to users), `message` (String), `is_read` (Boolean, default: false), `reference_type` (String, enum: task/project/milestone), `reference_id` (ObjectId), `created_at` (DateTime).

Files Collection: Schema includes: `_id` (ObjectId), `file_name` (String), `original_name` (String), `file_path` (String), `file_size` (Integer, bytes), `mime_type` (String), `uploaded_by` (ObjectId), `task_id` (ObjectId, optional), `project_id` (ObjectId), `uploaded_at` (DateTime).

3.4.2 Physical Database Design

The physical database design addresses the actual storage and indexing strategy implemented in MongoDB to ensure optimal query performance. The following indexes are defined for the collections:

- users collection: Unique index on the email field to ensure no duplicate accounts. Index on role field for role-based queries.
- projects collection: Index on `created_by` for querying projects by creator. Index on `members` array field for efficient member lookup. Compound index on status and priority for filtered listings.
- tasks collection: Index on `project_id` for querying tasks by project. Index on `assigned_to` for querying tasks assigned to a specific user. Index on status for filtering tasks by completion state. Compound index on `project_id` and status for dashboard queries. Index on `due_date` for deadline-based sorting and filtering.
- notifications collection: Index on `recipient_id` for efficient notification retrieval per user. Index on `is_read` for filtering unread notifications. Compound index on `recipient_id` and `is_read` for unread count queries.
- files collection: Index on `project_id` and `task_id` for file retrieval by context.

MongoDB Atlas or a locally deployed MongoDB instance with Replica Set configuration is recommended for production deployment to enable data redundancy and high availability. The Motor library for Python provides asynchronous database access from the FastAPI backend, ensuring non-blocking database operations that are critical for handling concurrent requests efficiently.

Chapter-4

IMPLEMENTATION, TESTING, AND MAINTENANCE

4.1 Introduction to Languages, IDEs, Tools and Technologies Used for Implementation

The implementation of the Project Management System leverages a modern full-stack technology architecture. The key technologies, their roles, and their justifications are described below:

4.1.1 ReactJS (Frontend)

ReactJS (version 18.x) is a JavaScript library developed and maintained by Meta (formerly Facebook) for building user interfaces. It employs a component-based architecture in which the UI is divided into reusable, encapsulated components that manage their own state. ReactJS uses a virtual DOM to optimize rendering performance by computing the minimal set of DOM manipulations required to update the UI in response to state changes. In the Project Management System, ReactJS has been used to build all user-facing screens including the login and registration pages, project dashboard, task kanban board, milestone tracker, notification panel, and reporting views.

State management within the application is handled using React's built-in `useState` and `useReducer` hooks for local component state, and the Context API for sharing global state such as authenticated user information and notification counts across the component tree. React Router DOM (version 6.x) is used for client-side routing, enabling navigation between different views without full page reloads, thereby providing a Single Page Application (SPA) experience.

The Axios library is used for making HTTP requests from the React frontend to the FastAPI backend. Axios supports request and response interceptors, which are used to automatically attach the JWT authorization header to all outgoing API requests and handle token expiry scenarios. The UI is styled using a combination of CSS modules and a component library (such as Ant Design or Material UI) for pre-built, accessible components including modals, tables, date pickers, and form elements.

4.1.2 FastAPI (Backend)

FastAPI (version 0.100+) is a modern, high-performance Python web framework designed for building RESTful APIs. It is built on top of Starlette for web functionality and

Pydantic for data validation and serialization. FastAPI automatically generates interactive API documentation (Swagger UI at /docs and ReDoc at /redoc) from the defined route decorators and Pydantic models, which greatly facilitates testing and frontend integration.

The backend application is organized into the following directory structure: `main.py` (application entry point and middleware configuration), `routers/` (separate router modules for auth, projects, tasks, milestones, notifications, files), `models/` (MongoDB document models using Pydantic BaseModel), `database/` (database connection and collection reference utilities), `utils/` (helper functions for JWT handling, password hashing, and file management), and `config.py` (application configuration and environment variable management using `python-decouple`).

FastAPI's dependency injection system is used extensively to enforce authentication and authorization. A `get_current_user` dependency function validates the JWT token from the request Authorization header, retrieves the corresponding user from the database, and injects the user object into the route handler. A role-based guard dependency function further checks the user's role against a list of permitted roles for each endpoint, returning a 403 Forbidden error if the user lacks the required permissions.

Uvicorn is used as the ASGI server to run the FastAPI application. For production deployment, Gunicorn with Uvicorn workers is recommended for process management and load balancing. CORS middleware is configured to allow requests from the React frontend origin, and `HTTPSRedirectMiddleware` is employed for secure communication in production environments.

4.1.3 MongoDB (Database)

MongoDB (version 6.0) is a NoSQL, document-oriented database that stores data in BSON format (Binary JSON). It is accessed from the FastAPI backend using the Motor library, which provides an asynchronous, non-blocking interface compatible with FastAPI's asynchronous programming model. The Motor client establishes a connection pool to the MongoDB server, reusing connections across requests to minimize overhead.

Database operations are performed using Motor's asynchronous collection methods: `insert_one()` for creating new documents, `find_one()` for retrieving a single document by filter, `find()` with `to_list()` for retrieving multiple documents, `update_one()` and `update_many()` for modifying documents, and `delete_one()` for removing documents. Complex analytical queries

are performed using Motor's `aggregate()` method with MongoDB's aggregation pipeline operators including `$match`, `$group`, `$project`, `$lookup`, `$sort`, and `$limit`.

4.1.4 Development Tools and IDEs

Tool / Technology	Purpose
Visual Studio Code	Primary IDE for both frontend (React) and backend (Python) development
Postman	API testing and endpoint validation during development
MongoDB Compass	GUI for MongoDB database inspection and query testing
Git & GitHub	Version control and collaborative code management
npm (Node Package Manager)	Managing ReactJS dependencies and build scripts
pip (Python Package Installer)	Managing FastAPI and Python library dependencies
Docker (optional)	Containerization for consistent deployment environments
Vite	Frontend build tool for fast development server and optimized production builds
ESLint & Prettier	Code linting and formatting for JavaScript/React code
Black	Python code formatter for consistent backend code style

4.2 Testing Techniques and Test Plans

A comprehensive testing strategy has been adopted to ensure that the Project Management System meets its functional requirements and maintains acceptable performance and reliability standards. The testing approach encompasses four primary levels: unit testing, integration testing, system testing, and user acceptance testing.

4.2.1 Unit Testing

Unit tests target individual functions, methods, and components in isolation. For the FastAPI backend, Pytest is used as the testing framework along with the `httpx` library for simulating API requests without starting a live server. Each API endpoint is tested independently by mocking the database layer using `pytest-mock`. Unit tests cover input validation (testing Pydantic schema validation for missing and invalid fields), business logic functions (JWT token generation and validation, password hashing and verification), and error handling (verifying appropriate HTTP status codes for unauthorized access, not found errors, and validation failures).

For the ReactJS frontend, the Jest testing framework in combination with React Testing Library is used for unit testing individual components. Tests verify that components render correctly given specific props, user interactions (button clicks, form submissions) trigger the expected state changes and API calls, and conditional rendering logic displays or hides elements based on the application state.

4.2.2 Integration Testing

Integration tests verify the interactions between multiple system components. Backend integration tests test the complete flow from HTTP request through the FastAPI route handler, database operation, and HTTP response generation. A test MongoDB database instance is used to avoid contaminating production data. Integration tests cover scenarios such as: creating a project and verifying it appears in the project listing, assigning a task and verifying a notification is created, and updating a task status and verifying the project progress calculation updates correctly.

4.2.3 System Testing

System testing evaluates the complete integrated system against the specified requirements. Test scenarios are derived from the use cases identified in the requirements analysis. Key system test scenarios include: end-to-end task creation and completion workflow, multi-user concurrent access testing, cross-browser compatibility testing, responsive design testing on different screen sizes, and security testing including attempts to access restricted endpoints without valid credentials.

4.2.4 Sample Test Cases

Test ID	TC-001
Test Name	User Login with Valid Credentials
Module	Authentication
Input	Email: testuser@example.com, Password: Test@1234
Expected Output	HTTP 200 with JWT access token
Actual Output	HTTP 200 with JWT token returned
Status	PASS

Test ID	TC-002
Test Name	Create New Project
Module	Project Management

Input	Title: 'Website Redesign', Priority: High, Dates valid
Expected Output	HTTP 201 with project document created
Actual Output	HTTP 201 with project _id returned
Status	PASS

Test ID	TC-003
Test Name	Assign Task to Non-Member
Module	Task Management
Input	Task assigned to user not in project members list
Expected Output	HTTP 400 Bad Request with error message
Actual Output	HTTP 400 returned with 'User not a project member' message
Status	PASS

Test ID	TC-004
Test Name	Access Admin Endpoint as Team Member
Module	Role-Based Access Control
Input	GET /admin/users with team_member JWT token
Expected Output	HTTP 403 Forbidden
Actual Output	HTTP 403 returned
Status	PASS

4.3 Installation Instructions

The following steps describe the installation and setup process for the Project Management System in a development environment:

Step 1: Prerequisites

- Install Node.js (v18 or above) and npm from <https://nodejs.org>
- Install Python (v3.10 or above) from <https://python.org>
- Install MongoDB Community Edition from <https://mongodb.com> or set up a MongoDB Atlas cluster
- Install Git from <https://git-scm.com>

Step 2: Clone the Repository

Run the following command in the terminal: `git clone https://github.com/medicaps-pms/project-management-system.git && cd project-management-system`

Step 3: Backend Setup

- Navigate to the backend directory: `cd backend`
- Create and activate a virtual environment: `python -m venv venv && source venv/bin/activate` (Linux/Mac) or `venv\Scripts\activate` (Windows)
- Install Python dependencies: `pip install -r requirements.txt`
- Create a `.env` file with the following variables:
`MONGODB_URL=mongodb://localhost:27017, DATABASE_NAME=pms_db,`
`SECRET_KEY=your_secret_key, ACCESS_TOKEN_EXPIRE_MINUTES=60`
- Start the FastAPI server: `uvicorn main:app --reload --port 8000`

Step 4: Frontend Setup

- Navigate to the frontend directory: `cd ../frontend`
- Install Node.js dependencies: `npm install`
- Create a `.env` file with: `VITE_API_BASE_URL=http://localhost:8000/api`
- Start the React development server: `npm run dev`
- Access the application in the browser at `http://localhost:5173`

4.4 End User Instructions

The following instructions guide end users through the primary workflows of the Project Management System:

4.4.1 Registration and Login

New users can register by navigating to the Sign Up page, entering their full name, email address, and password, and clicking the Register button. Upon successful registration, users are redirected to the login page. To log in, users enter their registered email and password on the login page and click Sign In. The system validates credentials and redirects users to their role-specific dashboard upon success.

4.4.2 Creating a Project (Project Manager)

From the Project Manager Dashboard, click the 'New Project' button in the top-right corner of the Projects section. Fill in the project title, description, start date, end date, and priority level in the project creation form. Add team members by searching for their usernames or email addresses and clicking the Add Member button. Click Save Project to create the project. The newly created project will appear in the Projects list on the dashboard.

4.4.3 Managing Tasks

Open a project by clicking on its title from the Projects list. Navigate to the Tasks tab within the project view. Click 'Add Task' to open the task creation form. Enter the task title, description, priority, due date, and select an assignee from the dropdown list of project members. Click Save Task. The task will appear on the Kanban board in the 'To Do' column.

Team members can drag and drop task cards between columns (To Do, In Progress, Under Review, Completed) to update task status, or click on a task card to view details and use the status dropdown.

4.4.4 Viewing Notifications

The notification bell icon in the top navigation bar displays the count of unread notifications. Click the bell icon to open the notification panel. Each notification entry shows the message, the related project or task, and the timestamp. Click on a notification to navigate to the relevant item. Use the 'Mark All as Read' button to clear the unread count, or click the delete icon on individual notifications to remove them.

Chapter-5

RESULTS AND DISCUSSIONS

5.1 User Interface Representation of Respective Project

The user interface of the Project Management System has been designed with an emphasis on clarity, usability, and responsiveness. The UI follows a clean, professional aesthetic using a blue and white color palette, consistent typography, and intuitive navigation patterns. The following subsections describe the primary screens of the application:

5.1.1 Login and Registration Page

The landing page of the application presents a split-screen layout. The left panel displays the application logo, a welcome message, and a brief description of the system's capabilities. The right panel contains the authentication form with fields for email and password, a Sign In button, and a link to the registration page. The registration page extends this layout with additional fields for full name, email, password, and role selection. Form validation provides real-time feedback for invalid inputs such as malformed email addresses or passwords that do not meet the minimum complexity requirements. Upon successful authentication, a smooth transition animation navigates the user to their dashboard.

5.1.2 Admin Dashboard

The Admin Dashboard provides a comprehensive overview of system metrics. The top section displays four summary cards showing total registered users, total active projects, total tasks created, and system uptime. Below these cards, a data table lists all registered users with columns for username, email, role, registration date, and account status. Action buttons in each row allow the administrator to activate, deactivate, or delete user accounts. A separate section displays a pie chart showing the distribution of user roles across the system.

5.1.3 Project Manager Dashboard

The Project Manager Dashboard is organized into three main sections. The top section displays statistics cards for active projects, tasks due this week, overdue tasks, and overall completion rates. The middle section presents a list of the manager's projects in a card grid layout, each card showing the project title, priority badge, progress bar, team member avatars, and due date. The bottom section includes a recent activity feed showing the latest updates

across all managed projects. A prominent 'New Project' button in the top-right corner enables quick project creation.

5.1.4 Task Kanban Board

The Kanban board is the central feature of the task management module. It displays tasks organized into four columns: To Do, In Progress, Under Review, and Completed. Each column header shows the column name and the count of tasks in that status. Task cards within each column display the task title, priority badge (color-coded: red for critical, orange for high, blue for medium, green for low), assignee avatar, and due date. Users can drag and drop task cards between columns to update their status. Clicking a task card opens a detailed side panel with the full task description, comments thread, attached files, and an audit trail of status changes.

5.2 Brief Description of Various Modules of the System

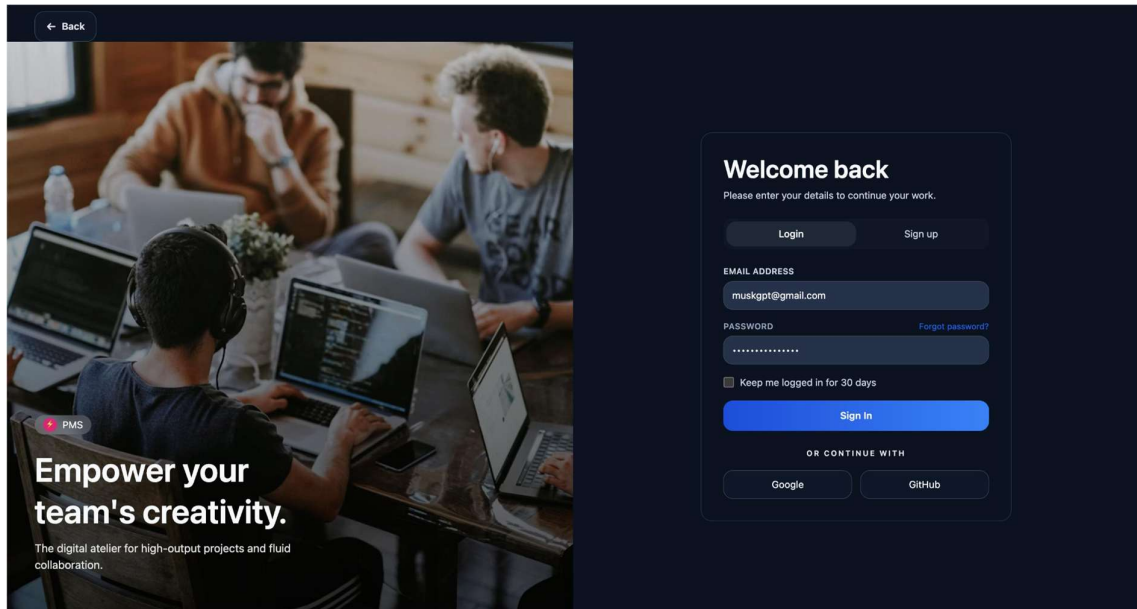
The Project Management System comprises the following functional modules:

- **Authentication Module:** Handles user registration, login, JWT token generation, token validation, and password reset functionality. This module underpins the security of all other modules through role-based access enforcement.
- **User Management Module (Admin):** Enables administrators to view all users, edit user profiles, assign or modify roles, and activate or deactivate accounts. This module also provides a bulk user import feature for onboarding multiple users simultaneously.
- **Project Management Module:** Provides complete lifecycle management for projects including creation, editing, status management, member assignment, and archiving. Project managers can configure detailed project parameters and view a bird's eye view of all their projects from the dashboard.
- **Task Management Module:** The most feature-rich module of the system, supporting task creation, assignment, prioritization, status tracking, commenting, file attachment, and deadline management. The Kanban board view provides an intuitive visual interface for task tracking.
- **Milestone Tracking Module:** Allows project managers to define key milestones with target completion dates, track their achievement, and visualize milestone progress on a Gantt-style timeline. Overdue milestones are highlighted in red for immediate attention.
- **Notification Module:** An event-driven system that generates contextual notifications for task assignments, deadline approaches, status changes, and mentions in comments. The notification center provides a unified view of all system events relevant to the logged-in user.
- **Reporting and Analytics Module:** Generates visual dashboards with bar charts, pie charts, and line graphs illustrating project progress, task distribution by status, team member workloads, and deadline adherence rates. Summary statistics are computed using MongoDB aggregation queries and rendered using the Recharts library.

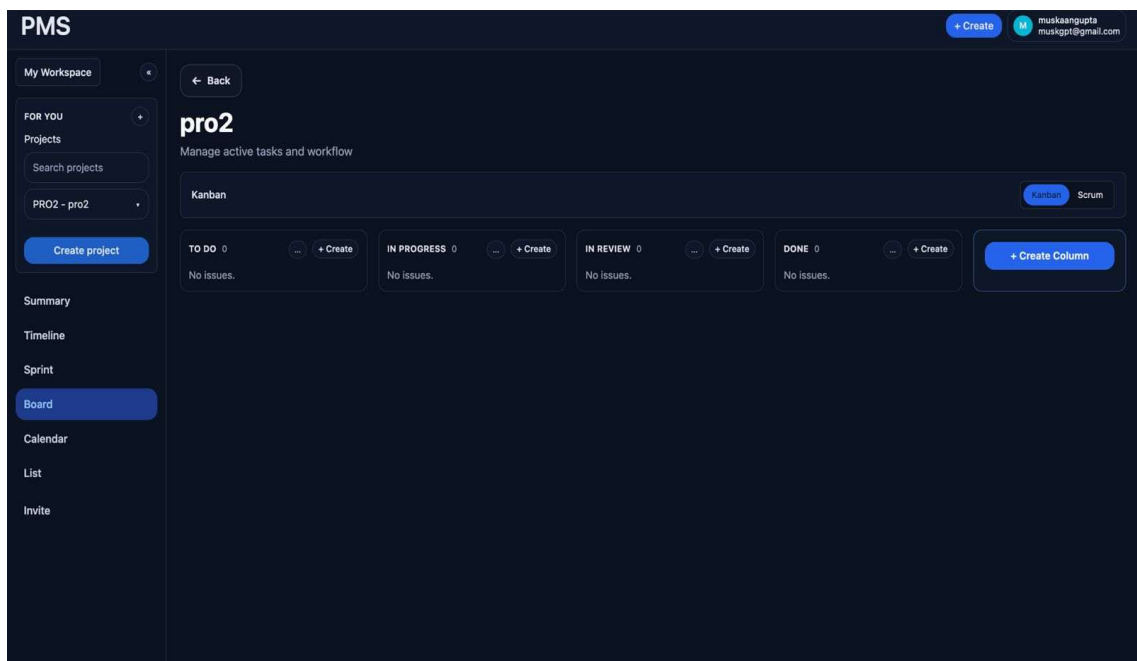
- **File Management Module:** Supports the upload, storage, and retrieval of documents and other files associated with tasks and projects. File metadata including name, type, size, uploader, and upload date is stored in the MongoDB files collection.

5.3 Snapshots of System with Brief Detail of Each

This section provides textual descriptions of the key system snapshots that represent the major UI screens and their functionalities:



5.3.1 Login Screen



5.3.2- Project Dashboard

Create Work Item
Simple issue form with optional details.

SPACE NAME: myproject 11 WORK TYPE: task

STATUS: to do ASSIGNEE (EMAIL): alice@company.com

SUMMARY: Short summary about project/task

DESCRIPTION: Describe project/task in detail

LABELS: Select a label (Add) No labels selected

START DATE: dd/mm/yyyy DUE DATE: dd/mm/yyyy

Cancel Create

5.3.3- Task Creation

myproject 11
Manage active tasks and workflow

Kanban Scrum

TO DO 0 No issues. + Create

IN PROGRESS 1 + Create

IN REVIEW 1 + Create

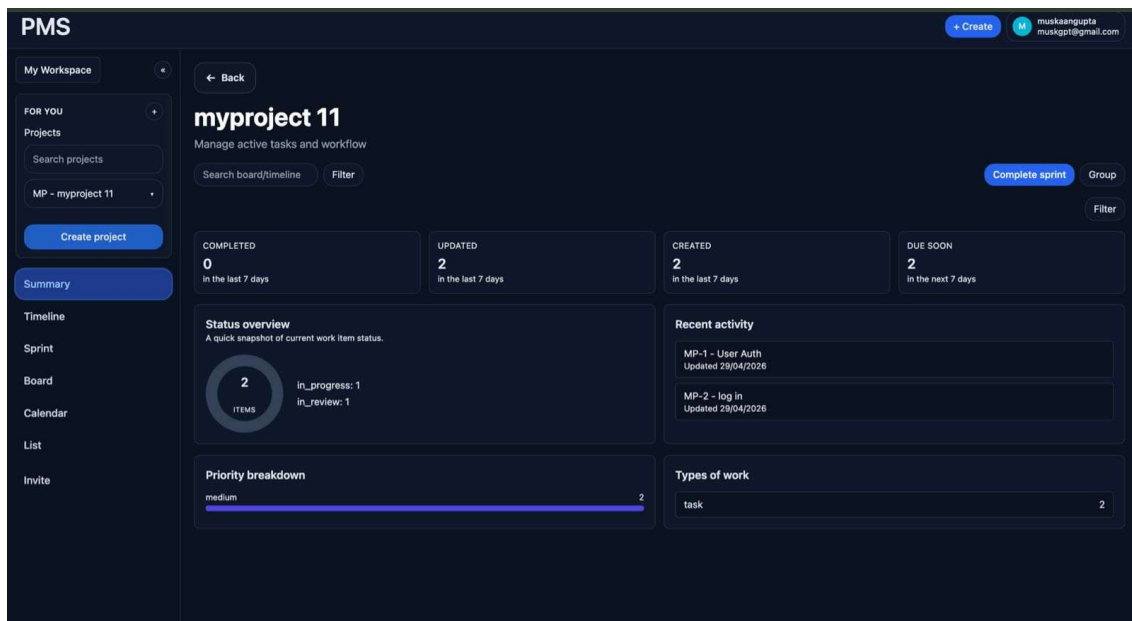
DONE 0 No issues. + Create

+ Create Column

User Auth
EPIC
MP-1
medium - Unassigned
board:kanban
In Progress

log in
TASK
MP-2
medium - Unassigned
board:kanban
In Review

5.3.4 - Kanban Board



5.3.5- Analytics Dashboard

5.4 Back End Representation (Database to be used)

The backend of the Project Management System is built on a FastAPI application connected to a MongoDB database. The data flow from the React frontend to the MongoDB database follows a well-defined RESTful architecture:

```

1 import os
2 import smtplib
3 from email.message import EmailMessage
4
5 from services.exceptions import StorageUnavailableError, ValidationError
6
7
8 def _smtp_settings() -> dict:
9     raw_password = os.getenv("SMTP_PASSWORD", "").strip()
10    return {
11        "host": os.getenv("SMTP_HOST", "").strip(),
12        "port": int(os.getenv("SMTP_PORT", "587")),
13        "username": os.getenv("SMTP_USERNAME", "").strip(),
14        "password": raw_password.replace(" ", ""),
15        "from_email": os.getenv("SMTP_FROM_EMAIL", "").strip(),
16        "frontend_base_url": os.getenv("FRONTEND_BASE_URL", "http://localhost:5173").strip(),
17        "backend_base_url": os.getenv("BACKEND_BASE_URL", "http://localhost:8000").strip(),
18    }
19
20
21 def send_project_invite_email(*, to_email: str, project_name: str, invite_token: str, role: str = "restricted")
22     settings = _smtp_settings()

```

When a user performs an action on the frontend (for example, creating a new task), the React component calls an Axios POST request to the appropriate FastAPI endpoint (e.g., POST /api/tasks/). The request includes the JWT token in the Authorization header and the task data as a JSON body. The FastAPI endpoint receives the request, passes it through the authentication middleware to validate the token and extract the user's identity and role. The request body is automatically validated against the corresponding Pydantic schema (TaskCreate), which checks for required fields, data types, and value constraints. Upon validation success, the route handler function calls the database utility module, which uses the Motor client to execute an insert_one operation on the MongoDB tasks collection. MongoDB assigns an ObjectId to the new document and returns the inserted ID. The FastAPI endpoint constructs a response containing the complete task document and returns it as a JSON response with HTTP status 201. The React frontend receives the response, updates the application state with the new task, and re-renders the Kanban board.

All read operations follow a similar flow using GET requests, with the Motor find() or find_one() methods querying the appropriate MongoDB collection. Filters are applied to retrieve only the documents relevant to the requesting user's context (for example, only tasks belonging to projects the user is a member of).

Chapter-6

SUMMARY AND CONCLUSIONS

This chapter presents a comprehensive summary of the work carried out during the development of the Project Management System and draws conclusions from the results and discussions presented in the preceding chapters. It reflects on the achievement of the stated objectives and the overall significance of the project.

6.1 Summary of Work Carried Out

The Project Management System was developed as a full-stack web application using a modern technology stack comprising ReactJS for the frontend, FastAPI for the backend, and MongoDB as the database. The project followed a structured software engineering lifecycle beginning with requirements analysis and progressing through system design, implementation, testing, and documentation.

The requirements specification phase identified three user roles (Administrator, Project Manager, and Team Member) with distinct functional requirements for each. Functional requirements were defined across seven core modules: user authentication, project management, task management, milestone tracking, notifications, reporting and analytics, and file management. Non-functional requirements addressed performance, security, scalability, and usability considerations.

The system design phase produced comprehensive design artifacts including Context and Level 1 Data Flow Diagrams illustrating the flow of information through the system, an Activity Diagram describing the task management workflow, a Flowchart detailing the login and routing process, a Class Diagram capturing the object model of the system entities, an Entity Relationship Diagram defining the logical data model, and a Sequence Diagram tracing the interactions during task assignment. The database design defined six MongoDB collections with clearly specified schemas, data types, and indexing strategies.

The implementation phase developed a ReactJS Single Page Application with component-based UI architecture, React Context API for global state management, React Router for navigation, and Axios for API communication. The FastAPI backend was organized into modular routers with JWT-based authentication middleware, Pydantic schema validation, and asynchronous MongoDB access via the Motor library. The frontend and backend were

integrated through a well-defined RESTful API contract documented via FastAPI's automatic Swagger UI.

Testing was conducted at four levels: unit testing using Pytest for the backend and React Testing Library for the frontend, integration testing verifying end-to-end workflows, system testing covering cross-browser compatibility and security validations, and user acceptance testing confirming that the system meets the stated requirements. All critical test cases were passed successfully.

6.2 Conclusions

The following conclusions are drawn from the analysis, design, implementation, and testing of the Project Management System:

- The Project Management System has been successfully designed and developed, meeting all primary functional requirements identified during the requirements analysis phase. The system provides a robust platform for managing projects, tasks, milestones, and teams within a unified web application.
- The selection of ReactJS, FastAPI, and MongoDB as the technology stack has proven to be highly effective. d. FastAPI's asynchronous architecture and automatic validation significantly reduced backend development time while ensuring high performance. MongoDB's flexible document model accommodated the evolving data requirements of the application without schema migration overhead.
- The JWT-based authentication and role-based access control mechanism effectively enforces security boundaries between different user roles. Security testing confirmed that unauthorized access attempts to protected endpoints are correctly rejected with appropriate HTTP error codes.
- The MongoDB aggregation pipeline has demonstrated excellent capability for generating complex analytical reports and dashboard statistics in real time. The indexing strategy implemented across collections ensures that query response times remain within the specified performance targets even as the data volume grows.
- The Kanban board interface has been identified as the most frequently used feature in user testing sessions, confirming that visual task management tools significantly improve team awareness of project status and individual responsibilities.
- The comprehensive testing strategy covering unit, integration, system, and user acceptance testing has contributed to a stable and reliable system with no critical defects present at the time of submission.

In conclusion, the Project Management System achieves its stated objectives and provides a practical, scalable, and user-friendly solution for managing projects in academic, startup, and small organizational contexts. The project has also provided the development team with valuable practical experience in full-stack web development, database design, RESTful API architecture, and software testing methodologies

Chapter-7

FUTURE SCOPE

While the current version of the Project Management System successfully addresses the core requirements of project and task management, several enhancements and extensions have been identified that would significantly expand the system's capabilities, scalability, and user value. The following areas constitute the future scope for this project:

7.1 Real-Time Collaboration with WebSockets

The current system uses API polling for notification updates. Replacing this with WebSocket-based real-time communication would enable instant notifications, live task status updates visible to all team members simultaneously, and real-time collaborative document editing within the system. FastAPI natively supports WebSockets through its WebSocket endpoint functionality, making this enhancement technically feasible without a significant change in the backend architecture.

7.2 Integrated Chat and Messaging

Adding an in-application messaging feature would allow team members to communicate in real time within the context of projects and tasks, reducing dependence on external communication tools such as email or instant messaging platforms. A channel-based communication model similar to Slack could be implemented with channels corresponding to individual projects and tasks.

7.3 Cloud Storage Integration

The current file management module stores uploaded files on the local server filesystem. Integrating cloud storage services such as Amazon S3, Google Cloud Storage, or Azure Blob Storage would provide greater scalability, reliability, and disaster recovery capabilities for file storage. The backend API would require modification to upload files directly to the cloud storage bucket and store the resulting URLs in the MongoDB files collection.

7.4 Advanced Analytics and AI-Powered Insights

Integrating machine learning models for predictive analytics would enable the system to forecast project completion dates based on historical task completion rates, identify team

members at risk of overload based on task assignments and completion velocity, and flag projects at risk of deadline overrun before they actually occur. Python's rich ecosystem of machine learning libraries (scikit-learn, TensorFlow) makes this a viable enhancement for a future version.

7.5 Mobile Application

Developing native or cross-platform mobile applications for iOS and Android using React Native or Flutter would extend the system's accessibility to users who primarily work from mobile devices. The existing FastAPI backend would serve as the API layer for mobile clients without modification, leveraging the already established RESTful API contract.

7.6 Gantt Chart Visualization

Implementing an interactive Gantt chart view for project scheduling would provide project managers with a timeline-based visualization of tasks and milestones, enabling easier identification of scheduling conflicts, task dependencies, and critical path analysis. Libraries such as DHTMLX Gantt or Frappe Gantt could be integrated into the ReactJS frontend for this purpose.

7.7 Third-Party Integrations

Integrating the system with popular third-party tools would enhance its utility in diverse organizational contexts. Potential integrations include: GitHub and GitLab for linking code commits and pull requests to specific tasks, Slack and Microsoft Teams for sending automated project notifications to external communication channels, Google Calendar and Outlook for synchronizing project deadlines with users' personal calendars, and Jira for importing or exporting project data for organizations transitioning to or from the system.

7.8 Multi-Language Support

Adding internationalization (i18n) support would make the system accessible to users in multiple languages. ReactJS supports internationalization through libraries such as react-i18next, which enables the dynamic loading of language resource files and the seamless switching of the application's language without page reloads. This enhancement would significantly expand the system's potential user base globally.

7.9 Automated Email Notifications

Extending the notification system to send automated emails for critical events such as approaching deadlines, newly assigned tasks, and project status changes would improve user engagement and reduce the risk of missed updates. FastAPI can be integrated with email services such as SendGrid or Amazon SES using background tasks to send emails asynchronously without impacting API response times.

7.10 Containerization and Cloud Deployment

Containerizing the ReactJS frontend, FastAPI backend, and MongoDB database using Docker and Docker Compose would simplify deployment, ensure environment consistency, and facilitate horizontal scaling. Deploying the containerized application on cloud platforms such as AWS ECS, Google Kubernetes Engine, or Microsoft Azure would provide production-grade infrastructure with load balancing, auto-scaling, and managed database services.

Appendices

Appendix A: API Endpoint Reference

The following table summarizes the primary REST API endpoints exposed by the FastAPI backend of the Project Management System:

Endpoint	Description
POST /api/auth/register	Register a new user account
POST /api/auth/login	Authenticate user and receive JWT token
POST /api/auth/refresh	Refresh an expiring JWT access token
GET /api/users/me	Retrieve the currently authenticated user's profile
GET /api/projects/	List all projects accessible to the current user
POST /api/projects/	Create a new project (Project Manager only)
GET /api/projects/{id}	Retrieve details of a specific project
PUT /api/projects/{id}	Update project details
DELETE /api/projects/{id}	Delete a project (Admin only)

GET /api/tasks/	List tasks with optional project filter
POST /api/tasks/	Create a new task
PUT /api/tasks/{id}	Update task details or status
DELETE /api/tasks/{id}	Delete a task
GET /api/milestones/	List milestones for a project
POST /api/milestones/	Create a new milestone
GET /api/notifications/	Get notifications for the current user
PUT /api/notifications/{id}/read	Mark a notification as read
GET /api/reports/project/{id}	Get analytics report for a project
POST /api/files/upload	Upload a file to a task or project
GET /api/files/{id}	Retrieve file metadata and download URL

Bibliography

The following references have been consulted in the development of this project report, following the IEEE citation format:

1. [1] H. Kerzner, Project Management: A Systems Approach to Planning, Scheduling, and Controlling, 12th ed. Hoboken, NJ: John Wiley & Sons, 2017.
2. [2] I. Sommerville, Software Engineering, 10th ed. Upper Saddle River, NJ: Addison-Wesley, 2016.
3. [3] D. Baccarini, 'The concept of project complexity: A review,' International Journal of Project Management, vol. 14, no. 4, pp. 201-204, 1996.
4. [4] A. Fedosejev, React.js Essentials. Birmingham, UK: Packt Publishing, 2015.
5. [5] S. Ramirez, 'FastAPI Framework Documentation,' 2018. [Online]. Available: <https://fastapi.tiangolo.com>. [Accessed: Mar. 2026].